**Title:** Linux Server Hardening Project
**Name:** R Ashwin
**Role:** Cybersecurity Enthusiast | SOC Analyst Aspirant
**Date:** 15.04.2026

# Linux Server Hardening & Brute-Force Attack Simulation

## Objective

To secure an Ubuntu Linux server against brute-force attacks by implementing industry-standard hardening techniques and validating their effectiveness through controlled attack simulation.

## Lab Architecture

- **Attacker Machine:** Kali Linux
- **Target Machine:** Ubuntu Server
- **Network Configuration:** NAT

## Initial State (Before Hardening)

The server was intentionally left in an insecure configuration:

- SSH running on default port (22)
- Password authentication enabled
- Root login permitted
- No firewall configured
- No brute-force protection mechanism

## Security Enhancements Implemented

1. SSH Hardening

- Configured SSH key-based authentication
- Disabled password authentication
- Disabled root login
- Configuration changes made in:

  `/etc/ssh/sshd_config`

```
# Authentication:

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile     .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to "no" here!
PasswordAuthentication no
#PermitEmptyPasswords no

# Change to "yes" to enable keyboard-interactive authentication.  Depending on
# the system's configuration, this may involve passwords, challenge-response,
# one-time passwords or some combination of these and other methods.
"/etc/ssh/sshd_config" 145L, 4289B written
ubuntu@ubuntu-server1:~$ sudo systemctl restart ssh
ubuntu@ubuntu-server1:~$ _
```

Figure 1: Disabled password authentication and root login.

2. Firewall Configuration

- Installed and configured UFW (Uncomplicated Firewall)
- Applied rules:
  - Deny all incoming traffic
  - Allow all outgoing traffic
- The firewall ensures that even if a service is misconfigured, unauthorized external access attempts are still restricted at the network level.
- This configuration ensures that only explicitly allowed services can communicate with the server, reducing exposure to network-based attacks

```
ubuntu@ubuntu-server1:~$ sudo ufw default deny incoming
[sudo: authenticate] Password:
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
ubuntu@ubuntu-server1:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
ubuntu@ubuntu-server1:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
ubuntu@ubuntu-server1:~$ sudo ufw enable
Firewall is active and enabled on system startup
ubuntu@ubuntu-server1:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)

ubuntu@ubuntu-server1:~$ _
```

Figure 2: Configured Firewall to deny incoming and allow outgoing traffic.

3. Fail2Ban Configuration

- Installed Fail2Ban to prevent brute-force attacks
- Configured SSH jail to monitor failed login attempts
- Automatically blocks malicious IP addresses after multiple failed attempts

```
ubuntu@ubuntu-server1:~$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable fail2ban
ubuntu@ubuntu-server1:~$ sudo systemctl start fail2ban
ubuntu@ubuntu-server1:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-03-02 15:45:31 UTC; 55s ago
 Invocation: 27844bb9f7bb4b0bbb590c77e7f97247
       Docs: man:fail2ban(1)
   Main PID: 3140 (fail2ban-server)
      Tasks: 5 (limit: 1803)
     Memory: 14.5M (peak: 14.6M)
        CPU: 236ms
     CGroup: /system.slice/fail2ban.service
             └─3140 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Mar 02 15:45:31 ubuntu-server1 systemd[1]: Started fail2ban.service - Fail2Ban Service.
Mar 02 15:45:31 ubuntu-server1 fail2ban-server[3140]: Server ready
ubuntu@ubuntu-server1:~$
```

Figure 3: Installed Fail2Ban on Ubuntu server.

```
ubuntu@ubuntu-server1:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
ubuntu@ubuntu-server1:~$ _
```

Figure 4: Displays current status of SSH jail.

## Key-Based Authentication Setup

- Generated SSH key pair on Kali Linux
- Copied public key to Ubuntu server
- Verified successful login using SSH key authentication

```
┌──(kali㉿kali)-[~]
└─$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kali/.ssh/id_rsa):
Enter passphrase for "/home/kali/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa
Your public key has been saved in /home/kali/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:rnnj4PCvHZESuwwaLqCTME/VLnTMy1drZUEo94AzPb4 kali@kali
The key's randomart image is:
+---[RSA 4096]----+
|          o oo    |
|     +  = *  .    |
|    o *  *.+o     |
|   o + + o.+.     |
|+ o o * S o.      |
|+* o + = oE       |
|= + . + o         |
| o   + =o.        |
|      *=+.        |
+----[SHA256]-----+
```

Figure 5: Generating SSH key pair on Kali Linux using ssh-keygen.

```
┌──(kali㉿kali)-[~]
└─$ ssh-copy-id ubuntu@192.168.32.131
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
ubuntu@192.168.32.131's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'ubuntu@192.168.32.131'"
and check to make sure that only the key(s) you wanted were added.


┌──(kali㉿kali)-[~]
└─$ ssh ubuntu@192.168.32.131
Welcome to Ubuntu 25.10 (GNU/Linux 6.17.0-14-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Mon Mar  2 14:32:48 UTC 2026

  System load:  0.0                Processes:              219
  Usage of /:   26.6% of 19.51GB   Users logged in:        0
  Memory usage: 22%                IPv4 address for ens33: 192.168.32.131
  Swap usage:   0%

 * Due to a now-resolved bug in the date command, this system may be unable
   to automatically check for updates. Manually install the update using:

     sudo apt install --update rust-coreutils

   https://discourse.ubuntu.com/t/enabling-updates-on-ubuntu-25-10-systems/

55 updates can be applied immediately.
To see these additional updates run: apt list --upgradable


Last login: Mon Mar  2 13:39:36 2026 from 192.168.32.130
ubuntu@ubuntu-server1:~$
```

Figure 6: Copying public key to Ubuntu server and testing key-based login.

## Attack Simulation

1. Before Hardening
    - Performed brute-force attack using Hydra
    - Password authentication allowed repeated login attempts
    - System was vulnerable to unauthorized access
    - The attack demonstrated how easily weak authentication mechanisms can be exploited.

```
ubuntu@ubuntu-server1:~$
ubuntu@ubuntu-server1:~$ sudo tail -f /var/log/auth.log
2026-03-02T13:42:13.550327+00:00 ubuntu-server1 sshd-session[2249]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.
168.32.130  user=ubuntu
2026-03-02T13:42:15.626561+00:00 ubuntu-server1 sshd-session[2249]: Failed password for ubuntu from 192.168.32.130 port 46432 ssh2
2026-03-02T13:42:17.181768+00:00 ubuntu-server1 unix_chkpwd[2252]: password check failed for user (ubuntu)
2026-03-02T13:42:18.839853+00:00 ubuntu-server1 sshd-session[2249]: Failed password for ubuntu from 192.168.32.130 port 46432 ssh2
2026-03-02T13:42:20.812305+00:00 ubuntu-server1 unix_chkpwd[2253]: password check failed for user (ubuntu)
2026-03-02T13:42:22.633573+00:00 ubuntu-server1 sshd-session[2249]: Failed password for ubuntu from 192.168.32.130 port 46432 ssh2
2026-03-02T13:42:23.707438+00:00 ubuntu-server1 sshd-session[2249]: Connection closed by authenticating user ubuntu 192.168.32.130 port 46432 [preauth]
2026-03-02T13:42:23.700281+00:00 ubuntu-server1 sshd-session[2249]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.32.13
0  user=ubuntu
```

Figure 7: System logs showing multiple failed SSH login attempts.

2. After Enabling Fail2Ban
    - Multiple failed login attempts triggered Fail2Ban
    - Attacker IP address was automatically banned
    - Brute-force attack was mitigated

```
"/etc/ssh/sshd_config" 145L, 4290B written
ubuntu@ubuntu-server1:~$ sudo systemctl restart ssh
ubuntu@ubuntu-server1:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     15
|  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
   |- Currently banned: 1
   |- Total banned:     1
   `- Banned IP list:   192.168.32.130
ubuntu@ubuntu-server1:~$
```

Figure 8: Fail2Ban detecting multiple failed login attempts and banning attacker IP.

## Post-Hardening Configuration

- Password authentication disabled
- Root login disabled
- Only SSH key-based authentication allowed

## Post-Hardening Attack Results

- Brute-force attack using Hydra failed completely
- Unauthorized login attempts blocked
- System remained secure under attack conditions

```
┌──(kali㉿kali)-[~]
└─$ hydra -l ubuntu -P ./rockyou.txt ssh://192.168.32.131
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-02 21:09:17
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort … (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 5487 login tries (l:1/p:5487), ~343 tries per task
[DATA] attacking ssh://192.168.32.131:22/
[ERROR] target ssh://192.168.32.131:22/ does not support password authentication (method reply 4).
```

Figure 9: Brute-force attack failing after security hardening.

## Security Analysis

A distributed brute-force attack would also fail because:

- Password-based authentication is completely disabled
- SSH access requires a valid private key
- Without possession of the private key, authentication cannot succeed

## Results

- Attacker IP successfully blocked
- Brute-force attack mitigated
- Server hardened effectively

## Security Principles Applied

- Defense-in-Depth
- Least Privilege
- Reduced Attack Surface
- Continuous Monitoring

## Key Learnings

- Importance of disabling password-based authentication
- Effectiveness of Fail2Ban in preventing brute-force attacks
- Role of firewalls in reducing attack surface
- Real-world simulation of attacker behavior using Hydra

## Conclusion

This project demonstrates how insecure default configurations can expose a system to brute-force attacks. By implementing layered security controls such as SSH hardening, firewall rules, and intrusion prevention systems, the server was successfully secured and attack attempts were effectively mitigated.

## Appendix: Commands Used

1. SSH Key Generation (Kali)
    - `ssh-keygen`

2. Copy Key to Server
    - `ssh-copy-id ubuntu@192.168.32.131`

3. SSH Configuration (Ubuntu)
    - `sudo vim /etc/ssh/sshd_config`
    - Changes made:
        - `PasswordAuthentication no`
        - `PermitRootLogin no`

4. Restart SSH
    - `sudo systemctl restart ssh`

5. Install UFW Firewall
    - `sudo apt install ufw`

6. Configure Firewall
    - `sudo ufw default deny incoming`
    - `sudo ufw default allow outgoing`
    - `sudo ufw enable`

7. Install Fail2Ban
    - `sudo apt install fail2ban`

8. Start Fail2Ban
    - `sudo systemctl enable fail2ban`
    - `sudo systemctl start fail2ban`

9. Check Fail2Ban Status
    - `sudo fail2ban-client status sshd`

10. Hydra Brute-Force Attack (Kali)
    - `hydra -l ubuntu -P ./rockyou.txt ssh://192.168.32.131`